

Packet Filtering Engine

Başak Balkan

Student No: 244410022

Kastamonu University

Computer Engineering

Abstract- This study discusses a Packet Filtering Engine application developed in C++ and with an object-oriented programming approach within the scope of the Security Tools category. In the developed system, network packets are represented according to source IP address, destination IP address, protocol, and port information; they are converted into an ALLOW or DENY decision with the help of predefined rules or rules added by the user. The application is built on the Packet, Rule, FilterEngine, Statistics, and Logger classes, so packet representation, rule matching, filtering decision, statistics tracking, and logging responsibilities are designed as separate modules. In addition, a web-based interface was prepared, and batch analysis could be performed on realistic packet records loaded from a CSV file. The test results show that the system produces the expected filtering decisions both in single-packet checking and in data-set-based analysis.

Keywords- packet filtering, security tools, firewall, C++, object-oriented programming, web interface, CSV analysis

I. INTRODUCTION

In network security applications, packet filtering is a fundamental approach that allows incoming or outgoing network traffic to be evaluated according to specific rules. This logic, which is used in firewall systems, is based on examining traffic through fields such as source address, destination address, protocol type, and port number. The NIST firewall guideline defines firewall systems as systems that control the flow of network traffic between networks or hosts with different security levels. The application developed in this study addresses this basic logic as an educational and OOP-centered software model.

The purpose of the study is not to produce a complex tool that captures real network packets live, but to model the concept of a packet filtering engine as an understandable, testable, and extensible C++ application. Therefore, packet data were obtained through user input and a CSV file, while filtering decisions were produced by rules defined within the application.

II. BACKGROUND AND TECHNOLOGIES USED

The packet filtering approach uses the basic header fields of IP and transport layer protocols. For IPv4, source and destination address information is among the fundamental components of the Internet Protocol structure defined in RFC 791. TCP is described in the current RFC 9293 document as one of the important transport layer protocols of the internet protocol stack. UDP is defined by RFC 768 as a datagram-based, simpler communication model. ICMP messages are also used to carry error and control messages over IP.

In the application, the C++ language was preferred because it supports class-based design and the encapsulation approach. On the web interface side, HTML, CSS, and JavaScript were used; the FileReader interface was utilized to read CSV files in the browser. The CSV format is a common text format that represents comma-separated tabular data and is documented by RFC 4180.

III. SYSTEM ARCHITECTURE

The system consists of two main layers. The first layer is the OOP core developed in C++. This core includes the Packet, Rule, FilterEngine, Statistics, and Logger classes. The second layer is the web interface that provides user-friendly interaction. When the application is run, the C++ starter code opens the index.html file in the default browser and verifies in the background that the OOP core works with a sample filtering operation. In the Windows environment, the ShellExecuteA function was used for opening the file.

In this architecture, rules are kept in memory as a list. Each packet is compared with the rules in order, and the decision of the first matching rule is applied. If no rule matches, the system produces DENY as the default result. This choice is consistent with the default blocking approach often used in security tools, because undefined or unexpected traffic is not considered allowed.

A. Class Responsibilities

The Packet class stores source IP, destination IP, protocol, and port information. The Rule class determines under which conditions a rule will match a packet. The FilterEngine class stores the rules and produces the decision for the incoming packet. The Statistics class tracks the total, allowed, and blocked packet counts. The Logger class records filtering results in the log.txt file.

IV. IMPLEMENTATION

A. Packet and Rule Model

In the application, each packet is represented with the sourceIP, destinationIP, protocol, and port fields. Rules consist of the action, protocol, sourceIP, destinationIP, and port fields. When the value ANY is used in rule fields, the related field matches all values. In the port field, the value -1 is used as the equivalent of ANY. This approach provides a simple but understandable rule matching mechanism.

B. Filtering Engine

The FilterEngine class stores rules in a vector structure and tests the packet against all rules in order with the evaluate function. The ALLOW or DENY value of the first matching rule is returned as the result. This structure is a simplified counterpart of the rule-order logic seen in real firewall systems.

C. Web Interface and CSV Analysis

The web interface includes sections for single-packet checking, adding a new rule, displaying existing rules, showing statistics, listing logs, and CSV-based batch analysis. In the CSV upload section, when the user selects the packets.csv file, the browser reads the file content as text, each line is converted into a packet object, and it is analyzed with the same rule matching logic. In this way, the application is made capable of working not only with manual input but also with realistic data sets.

D. Logging and Statistics

At the end of each analysis, the total number of packets checked, the number of allowed packets, and the number of blocked packets are updated. On the C++ side, the Logger class writes the result to the log.txt file, while on the web interface side the log records are listed on the page. This structure represents the concept of traceability, which is important in security tools, at a simple level.

V. TEST SCENARIOS AND RESULTS

The application was verified through three basic test groups. First, in single-packet checking, when the source IP address 192.168.1.10, destination IP address 10.0.0.5, TCP protocol, and port 80 were used, the system produced an ALLOW result. This result is consistent with the ALLOW TCP 192.168.1.10 ANY 80 rule among the default rules.

In the second test, a DNS-like packet was entered by using the UDP protocol and port 53, and the system produced a DENY result. This result shows that the DENY UDP ANY ANY 53 rule works correctly. In the third test, the eight packets in the packets.csv file were analyzed in batch, and the results were shown in table form. During CSV analysis, it was observed that the statistics values and log records were updated.

VI. DISCUSSION

The developed system is not a full-fledged security product that captures packets from a live network card. In contrast, it is an educational application that models the logic of a packet filtering engine at the software level and makes OOP principles visible. Capturing live traffic may require additional drivers such as Npcap, administrator privileges, and more detailed protocol parsing operations. In this study, the CSV-based data-set approach was preferred because it is stable and portable.

The strong aspect of the application is that the filtering decision is modeled with separate classes and clear responsibilities. Through encapsulation, packet and rule data are kept inside classes, and decision production is separated into the FilterEngine class. Thus, the system can be improved in the future with features such as IP range support, port range support, rule priority, timestamp, or protocol extensions.

VII. LIMITATIONS AND FUTURE WORK

The current application compares IP addresses through textual equality; it does not perform CIDR range or subnet mask calculation. In addition, advanced fields such as packet content, TCP flags, or connection state are not evaluated. In future work, features such as IP range support, port range definition, rule priority, CSV export, and more detailed reporting may be added. At a more advanced stage, integration can be established with a library capable of capturing real network traffic.

VIII. CONCLUSION

In this study, a Packet Filtering Engine application suitable for the Security Tools category was developed by using C++ and the object-oriented programming approach. The application evaluates network packets according to source IP, destination IP, protocol, and port information and produces an ALLOW or DENY decision according to the defined rules. The web interface and CSV-based batch analysis support have made the use of the system more understandable and realistic. The obtained results show that the developed application correctly models packet filtering logic at a basic level and demonstrates OOP principles within the project in an applicable manner.

REFERENCES

- [1] K. Scarfone and P. Hoffman, "Guidelines on Firewalls and Firewall Policy," NIST Special Publication 800-41 Revision 1, National Institute of Standards and Technology, 2009. [Online]. Available: <https://csrc.nist.gov/pubs/sp/800/41/r1/final>
- [2] J. Postel, "Internet Protocol," RFC 791, Internet Engineering Task Force, 1981. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc791>
- [3] M. Eddy, Ed., "Transmission Control Protocol (TCP)," RFC 9293, Internet Engineering Task Force, 2022. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc9293>
- [4] J. Postel, "User Datagram Protocol," RFC 768, Internet Engineering Task Force, 1980. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc768>
- [5] J. Postel, "Internet Control Message Protocol," RFC 792, Internet Engineering Task Force, 1981. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc792>
- [6] Y. Shafranovich, "Common Format and MIME Type for Comma-Separated Values (CSV) Files," RFC 4180, Internet Engineering Task Force, 2005. [Online]. Available: <https://datatracker.ietf.org/doc/html/rfc4180>
- [7] Microsoft, "ShellExecuteA function (shellapi.h)," Microsoft Learn. [Online]. Available: <https://learn.microsoft.com/en-us/windows/win32/api/shellapi/nf-shellapi-shellexecutea>
- [8] MDN Web Docs, "FileReader: readAsText() method," Mozilla Developer Network. [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/API/FileReader/readAsText>
- [9] B. Stroustrup, The C++ Programming Language, 4th ed. Boston, MA, USA: Addison-Wesley, 2013.
- [10] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Boston, MA, USA: Addison-Wesley, 1994.
- [11] W. R. Stevens, TCP/IP Illustrated, Volume 1: The Protocols. Boston, MA, USA: Addison-Wesley, 1994.
- [12] M. Sikorski and A. Honig, Practical Malware Analysis. San Francisco, CA, USA: No Starch Press, 2012.